Container Foundations

Docker Student Exercises

Ubuntu / WSL Bash · Original Nginx guided exercises

Day 1 · Student follow-along

Use this guide **after the instructor has explained entry to the Workshop lab machine**. In Windows Terminal on the lab machine, select the **Ubuntu** profile (Ubuntu/WSL **Bash**), not the default PowerShell. Do not enter these Bash commands in PowerShell or on your personal computer.

> **How to use the exercises**
> Complete the local steps in **D0–D8** in order and keep the same terminal. Plain command blocks run on the **lab Docker host**; D2 clearly labels the two lines entered inside a container. Copy a complete block without adding shell prompts. A trailing backslash continues a long command; heredoc closing markers must stand alone.

Full containers Day 1 agenda · Eastern · 08:00–15:15

Time	Session
08:00–08:30	Workshop lab and Docker terminal readiness
08:30–09:30	Chapters 1–2: container concepts, image trust and first Nginx run
09:30–09:45	Break
09:45–11:00	Chapter 3: lifecycle, inspection, access and ports
11:00–11:15	Break
11:15–12:45	Chapter 3 storage; Chapter 4 registries and image management
12:45–13:45	Lunch
13:45–15:15	Chapter 5: Dockerfile build, guided practice, recap and AKS handoff

Core vs. optional: all local Nginx exercises need no registry account. D3 includes an optional read-only bind mount. Shaded instructor-only blocks cover approved private-registry access, snapshots and publishing; they are not student completion requirements.

> **Lab validation status**
> Commands have been reviewed for Bash syntax, but **have not been runtime-rehearsed on the class lab**. Every outcome in this guide is labelled EXPECTED, not a captured test log. The instructor must complete the pre-class checks at the end before class.

Introducing Container Technology

Concept essentials and terminal readiness

Container essentials and Docker architecture

An image contains filesystem/runtime files, layers and metadata—not running processes or the host kernel. A container is an image instance whose processes use the Docker host's kernel; its writable layer is separate. The Docker CLI talks to an API/server daemon. Namespaces separate resource views, cgroups manage resource use and seccomp filters system calls; these controls do not guarantee complete isolation or safety.

D0 · Get the Docker terminal ready

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

1 · Use the lab machine, not your personal computer

In **Windows Terminal**, choose the **Ubuntu** profile (not the default PowerShell). Run the checks below in that Ubuntu/WSL Bash shell. Docker Desktop may supply the Linux engine in the background; no PowerShell commands are needed.

```
docker version
docker info --format "{{.OSType}}"
docker context show
```

EXPECTED · Client and Server information is returned; the engine OS is linux.

PAUSE / SAFETY · If the Docker CLI is missing, WSL cannot reach the Server, the OS is not linux, or permissions fail, pause and ask the instructor. Do not install Ubuntu/WSL or Docker, or change privileges, daemon settings or WSL integration.

2 · Create a dedicated working folder

```
mkdir -p "$HOME/container-foundations"
cd "$HOME/container-foundations"
export BASE_IMAGE='docker.io/library/nginx:alpine'
pwd
```

EXPECTED · The working folder is your own container-foundations directory and BASE_IMAGE identifies the class image.

WHY THIS MATTERS · Keep this terminal. In a new terminal, return to this folder and re-enter the BASE_IMAGE export above. The instructor must validate the selected alpine tag before class; it is a moving tag, not an immutable version.

Folder rule: D1–D6 use the class folder. D7 enters its relative `custom-web` subfolder. Relative files stay inside the class folder; no installation or privilege changes are part of this guide.

Q1 · Optional interaction checkpoint

Which three kinds of applications can be packaged as containers for immediate consumption? (Choose three.)
a. A virtual machine hypervisor
b. A blog software, such as WordPress
c. A database
d. A local file system recovery tool
e. A web server
Optional answers are at the end.

Pull is not run; provenance is not a safety guarantee

The class base is `docker.io/library/nginx:alpine`, the Docker Official Image namespace on Docker Hub—not Quay. Pull stores an image locally; run creates and starts an instance. Official/custom provenance must still be evaluated alongside publisher, recipe and scanning.

D1 · Fetch and run the Nginx container

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

Before you start: D0 complete; the Docker Linux server is reachable.

1 · Pull the selected image

```
docker pull "$BASE_IMAGE"
docker images docker.io/library/nginx
```

EXPECTED · Docker downloads the image or reports that the local copy is up to date; the image appears in the list.

PAUSE / SAFETY · Public Docker Hub pulls depend on classroom network/access and rate limits. The instructor must verify or pre-pull the selected image. If blocked, pause for an instructor-provided approved registry copy; do not invent a destination or create a personal account.

2 · Run a named web server with a non-secret environment variable

```
docker run -d --name cf-web -p 127.0.0.1:8080:80 -e COURSE_MODE=demo "$BASE_IMAGE"
docker ps
docker exec cf-web printenv COURSE_MODE
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8080
```

EXPECTED · cf-web is running, printenv returns demo, and curl returns the Nginx welcome page.

WHY THIS MATTERS · -d detaches; --name identifies this instance; -p publishes host loopback 8080 to container port 80. -e belongs to run, not pull. COURSE_MODE is a non-secret value inspected here; it does not automatically reconfigure Nginx.

PAUSE / SAFETY · Run curl on the Docker host inside the lab. A personal-laptop localhost is a different machine. If a name or port is already in use, identify your own resource with the instructor rather than deleting other lab resources.

Docker, Podman and rootless modes

Docker uses a client/server engine; Podman typically manages containers without a central daemon. Rootless operation changes the user/privilege model and has configuration constraints. Neither tool nor rootless mode makes images automatically safe. Use only the instructor's approved Docker configuration here.

HTTP checkpoint behavior

The shared curl checks retry briefly when a detached service is still starting. A successful response is the checkpoint—not proof that every later request will succeed. If retries are exhausted, pause with the instructor and inspect the selected container; do not change privileges or remove other resources.

D2 · Observe and manage the container life cycle

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

Before you start: D1 created the cf-web instance.

Life cycle: create prepares an instance; start runs it; stop retains its writable layer; rm removes that instance. A new run creates a different container.

1 · Observe the application

```
docker ps
docker ps -a
docker port cf-web
docker logs --tail 10 cf-web
docker inspect --format "{{.State.Status}}" cf-web
docker exec cf-web nginx -v
```

EXPECTED · The lists show cf-web, the port mapping is visible, and logs/status/version answer different questions.

ps = running; ps -a = all states; logs = application output; inspect = state/configuration; exec = an extra process.

2 · Stop and restart the same instance

```
docker stop cf-web
docker ps
docker ps -a
docker start cf-web
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8080
docker restart cf-web
```

EXPECTED · After stop, cf-web leaves the running list but remains in ps -a; start and restart reuse the existing instance.

3 · Use a shell, then remove only the stopped exercise container

LAB HOST — open the container shell

```
docker exec -it cf-web sh
```

INSIDE CONTAINER — type ls, then exit

```
ls /usr/share/nginx/html
exit
```

BACK ON LAB HOST — after exit returns

```
docker stop cf-web
docker rm cf-web
docker images docker.io/library/nginx
```

EXPECTED · exit returns you to the host terminal; cf-web is removed, while the image remains locally.

Shell boundary: only ls and exit above run inside the container. exit ends that shell—not the host terminal. This image includes sh; not every image does.

4 · Separate creation from starting

```
docker create --name cf-created -p 127.0.0.1:8080:80 "$BASE_IMAGE"
docker ps -a
docker start cf-created
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8080
docker stop cf-created
docker rm cf-created
```

EXPECTED · create leaves a Created instance; start runs it; the page is reachable only after starting.

D3 · Attach persistent storage to Nginx

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

Before you start: D2 has released port 8080; port 8081 and the cf-volume names are available.

Storage reminder: stop/start retains a container writable layer; removal loses it. A named volume has a separate lifetime. An empty volume receives image-directory content by default. A bind mount instead exposes an existing Docker-host path and hides image content at that mount point.

1 · Create and attach a named volume

```
docker volume create cf-html
docker run -d --name cf-volume -p 127.0.0.1:8081:80 --mount \
  type=volume,src=cf-html,dst=/usr/share/nginx/html "$BASE_IMAGE"
docker exec cf-volume sh -c \
  'printf "%s\n" "<h1>Persistent Nginx page</h1>" > /usr/share/nginx/html/index.html'
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8081
```

EXPECTED · The response contains Persistent Nginx page.

2 · Replace the container, retain the data

```
docker stop cf-volume
docker rm cf-volume
docker run -d --name cf-volume-next -p 127.0.0.1:8081:80 --mount \
  type=volume,src=cf-html,dst=/usr/share/nginx/html "$BASE_IMAGE"
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8081
docker stop cf-volume-next
docker rm cf-volume-next
```

EXPECTED · The replacement serves the same Persistent Nginx page although the first container no longer exists.

KEEP cf-html: retain this volume until D8. Deleting it deletes the persistent exercise page.

OPTIONAL · STUDENT FOLLOW-ALONG ONLY IF THE INSTRUCTOR APPROVES HOST-PATH ACCESS

3 · Optional: mount your host content read-only

```
cd "$HOME/container-foundations"
mkdir -p site
printf '%s\n' '<h1>Read-only bind-mounted page</h1>' > site/index.html
docker run -d --name cf-bind -p 127.0.0.1:8081:80 --mount \
  "type=bind,src=$PWD/site,dst=/usr/share/nginx/html,readonly" "$BASE_IMAGE"
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8081
docker stop cf-bind
docker rm cf-bind
```

EXPECTED · Nginx serves the file from your site folder; container removal does not remove that host file.

Read-only: the container cannot modify this mounted site folder. If host-path access is not validated, skip this optional step. The printf format contains the literal backslash-n characters shown in the command.

Ports and the right machine

The left side of a published mapping is the Docker-host endpoint; the right side is the container's application port. Host and container port numbers need not match. Image EXPOSE metadata documents a port; it does not publish it. A personal computer's localhost is not the Workshop lab host.

D4 · Map ports and access the application

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

Before you start: Docker-host port 8082 is available.

1 · Use a different host port

```
docker run -d --name cf-ports -p 127.0.0.1:8082:80 "$BASE_IMAGE"
docker port cf-ports
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8082
docker exec cf-ports sh -c "head -n 3 /usr/share/nginx/html/index.html"
docker stop cf-ports
docker rm cf-ports
```

EXPECTED · The host mapping is 8082 to container 80; curl reaches the same application through the new host port.

WHY THIS MATTERS · The application still listens on port 80 inside the container; its lab-host endpoint is now 127.0.0.1:8082. exec starts a process in the selected running container; it is not SSH.

Choose the observation tool for the outcome

curl: check the HTTP service from the Docker host. **logs:** inspect application stdout/stderr. **exec:** inspect files or run a supported diagnostic inside the running instance. **inspect / port:** read state/configuration and published mappings. These answer different questions.

State at the end of Chapter 3

D2 and D4 removed their dedicated containers; D3 removed its container instances but retained `cf-html`. If the optional bind mount ran, its host file remains in `site/index.html`. The downloaded base image remains available for the next chapters.

Registry selection, visibility and trust

Public repositories may allow anonymous pulls; **private** repositories require authorization. A fully qualified image name selects its registry, namespace/repository and tag. Omitting the registry defaults to Docker Hub—not an ordered search of registries. Official or verified provenance is useful but not a vulnerability-free guarantee; custom/private images can be appropriate when provenance and controls are verified.

Access and discovery: Docker uses HTTPS/TLS trust and its approved credential store/helper. Do not disable TLS or put credentials in files, command literals or URLs. docker search searches Docker Hub only; Quay discovery uses its repository interface or vendor API. Authentication, enumeration and search support depend on the registry.

Registry HTTP API · concept only	What the endpoint can indicate
`GET /v2/`	Registry API entry point; authentication may be required.
`GET /v2/_catalog`	Repository enumeration; vendor/auth permissions may limit or omit it.
`GET /v2/<repository>/tags/list`	Tags for a repository when supported and authorized.

No API fetch is required: these are illustrative paths, not commands. Do not assume Quay supports global anonymous catalog listing. Docker login does not authenticate a separate curl request; the CLI handles its own registry authentication.

D5 · Find images and inspect registry access

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

1 · Search Docker Hub, then inspect the exact image

```
docker search --filter is-official=true --limit 5 nginx
docker pull "$BASE_IMAGE"
docker images docker.io/library/nginx
docker image inspect --format "{{.Id}}" "$BASE_IMAGE"
docker image inspect --format "{{json .RepoDigests}}" "$BASE_IMAGE"
```

EXPECTED · Search identifies official Docker Hub results, and inspection shows the local image identity and available repository digests.

Local image store: pull downloads layers/metadata. Image IDs describe local content; tags are mutable names; repository digests identify repository content when available. Listing local images is not a registry search.

OPTIONAL ● INSTRUCTOR ONLY — students skip this block

2 · Instructor-only: authenticate to an approved Quay repository

```
docker login quay.io
```

EXPECTED · The login flow succeeds only for an authorized account and credential method.

Students do not sign up or log in. Use only an approved training account and credential method at the Docker prompt or approved helper. Do not display credentials. Quay login is separate from Docker Hub access.

D6 · Manipulate Nginx images

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

Before you start: D5 local search/inspection is complete. Archive steps are local; snapshots and publishing are instructor-only options.

> **Image references and archive scope**
> A tag adds a name without rebuilding content. Removing one extra tag need not remove another tag for the same image. Docker save writes an ordinary TAR image archive with layer/metadata layout; load imports that archive. This is not a generic filesystem backup and does not back up container changes or named-volume data.

1 · Add a tag, archive the image, then load it

```
cd "$HOME/container-foundations"
docker tag "$BASE_IMAGE" cf-nginx:archive
docker image inspect --format "{{.Id}}" "$BASE_IMAGE"
docker image inspect --format "{{.Id}}" cf-nginx:archive
docker save -o nginx-training.tar cf-nginx:archive
docker image rm cf-nginx:archive
docker load -i nginx-training.tar
docker images cf-nginx
```

EXPECTED · Both references initially have the same image ID. save writes an image archive; load restores the archive tag.

WHY THIS MATTERS · nginx-training.tar is relative to the class working folder. Removing cf-nginx:archive here leaves the base reference intact; load restores the archive tag. Archive details can be instructor-led depth, but no account or compression flag is required.

OPTIONAL DEPTH · INSTRUCTOR ONLY — filesystem diff and commit

2 · Instructor-only optional: diff and commit a changed instance

```
docker run -d --name cf-commit "$BASE_IMAGE"
docker exec cf-commit sh -c \
  'printf "%s\n" "<h1>Temporary committed page</h1>" > /usr/share/nginx/html/index.html'
docker diff cf-commit
docker commit --message "Class snapshot only" cf-commit cf-nginx:snapshot
docker images cf-nginx
docker stop cf-commit
docker rm cf-commit
```

EXPECTED · diff reports filesystem changes and commit produces a new local image snapshot.

diff markers: A = added, C = changed, D = deleted. commit captures container filesystem/configuration, not mounted volume data. A snapshot can retain unwanted runtime files; the reproducible build method follows in Chapter 5.

PAUSE / SAFETY · Do not snapshot production containers or include secrets. This optional class snapshot is not the recommended application-build workflow.

OPTIONAL · INSTRUCTOR ONLY BY DEFAULT — not a student account requirement

D6 · Manipulate Nginx images (continued)

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations`

> **Authorization gate — read before any registry command**
>
> Only proceed after the D5 login demonstration and confirmation of write permission to an **existing approved PRIVATE training repository**. The namespace and repository prompts collect non-secret destination names, not passwords. Never use another person's namespace or publish publicly. If authorization or private visibility is not confirmed, discuss this block without running it and continue to D7.

3 · Optional: push a class mirror to your own approved private Quay repository

```
read -r -p 'Your authorized Quay namespace: ' QUAY_NAMESPACE
read -r -p 'Existing approved private repository name: ' QUAY_REPOSITORY
export QUAY_IMAGE="quay.io/${QUAY_NAMESPACE}/${QUAY_REPOSITORY}:training-nginx"
docker tag "$BASE_IMAGE" "$QUAY_IMAGE"
docker push "$QUAY_IMAGE"
docker pull "$QUAY_IMAGE"
```

EXPECTED · The authorized repository receives the tagged image and Docker can pull that reference.

WHY THIS MATTERS · You are publishing your own repository copy of the selected base image. The Quay repository is a mirror/custom repository, not the Docker Official Image namespace. Push transfers image content, not a running container or its persistent data.

> **What this push does—and does not—publish**
>
> The prompted Quay destination is a mirror/custom repository copy of the selected Docker Hub base, **not an official Quay Nginx namespace**. Push transfers image content and references; it does not move a running container, its writable runtime state or its volume data. Credentials remain at the approved login/helper interface and must never appear in command blocks, URLs or screenshots.

> **Optional variables and continuing the class**
>
> `QUAY_NAMESPACE` and `QUAY_REPOSITORY` are used again only in D7's optional publishing block. Students who skip this page can complete every local core exercise. D8 logout ends local registry authentication but does not delete any remote repository content.

Design and choose the parent image

Start from a reviewed Dockerfile/base with known provenance and a small, intentional build context. Reusing a Dockerfile still requires reviewing its parent, copied files and runtime behavior. The class uses the same `docker.io/library/nginx:alpine` parent and keeps source files in custom-web. Never include secrets.

Q2 · Optional interaction checkpoint

What are two typical scenarios for creating a Dockerfile to build a child image from an existing image? (Choose two.)
a. Adding new runtime libraries.
b. Setting constraints on a container's access to the host machine's CPU.
c. Adding internal libraries to be shared as a single image layer by multiple container images for different applications.
Optional answers are at the end.

D7 · Build a custom Nginx image with a Dockerfile

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations → custom-web`

Before you start: D0–D6 local core activities complete. Docker-host port 8083 is available.

1 · Create the build context and web page

```
cd "$HOME/container-foundations"
mkdir -p custom-web
cd custom-web
cat > index.html <<'HTML'
<!doctype html>
<html><body><h1>Hello containers - v1</h1></body></html>
HTML
cat > Dockerfile <<'DOCKERFILE'
FROM docker.io/library/nginx:alpine
LABEL purpose="Container foundations training"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
DOCKERFILE
cat Dockerfile
```

EXPECTED · The custom-web folder contains Dockerfile and index.html.

Bash heredocs: enter each complete block. The closing HTML and DOCKERFILE markers must appear alone at the start of their own lines. Do not put a shell prompt into either file; these blocks are not PowerShell.

Recipe intent: FROM selects the parent; LABEL records metadata; COPY adds the page; EXPOSE documents port 80; CMD supplies the default foreground Nginx command. The parent image's ENTRYPOINT is inherited.

D7 · Build a custom Nginx image with a Dockerfile (continued)

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations/custom-web`

2 · Build, run and test version 1

```
docker build -t cf-custom:v1 .
docker run -d --name cf-custom-web -p 127.0.0.1:8083:80 cf-custom:v1
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8083
docker logs --tail 10 cf-custom-web
```

EXPECTED · The page contains Hello containers - v1.

WHY THIS MATTERS · The final dot is the current build context. Build creates an image; run creates a container; curl checks the published HTTP service.

3 · Change the source and replace the container

```
printf '%s\n' '<!doctype html><html><body><h1>Hello containers - v2</h1></body></html>' \
  > index.html
docker build -t cf-custom:v2 .
docker stop cf-custom-web
docker rm cf-custom-web
docker run -d --name cf-custom-web -p 127.0.0.1:8083:80 cf-custom:v2
curl -fsS --retry 10 --retry-connrefused --retry-delay 1 http://127.0.0.1:8083
```

EXPECTED · The replacement returns Hello containers - v2.

WHY THIS MATTERS · Changing source files does not change a previously built image or existing container. Restarting the old instance does not switch it to a newly built tag.

Instructions, defaults and cache—compact reminders

CMD supplies default command/arguments; ENTRYPOINT defines the invocation and is not universally present. COPY copies build-context files; ADD has extra archive/URL behavior, so use COPY for this page. RUN executes during build; WORKDIR selects a directory; ENV sets non-secret defaults; USER selects the runtime user. Filesystem-changing instructions can add layers; metadata-only instructions do not all add filesystem layers. Changed inputs can invalidate later cache steps.

Q3 · Optional interaction checkpoint

Which method for creating container images is recommended by the containers community?
a. Run commands inside a basic OS container, commit the container, and then save or export it as a new container image.
b. Run commands from a Dockerfile and push the generated container image to an image registry.
c. Create the container image layers manually from tar files.
d. Run the docker build command to process a container image description in YAML format.
Optional answers are at the end.

OPTIONAL · INSTRUCTOR ONLY unless the learner has an approved writable private repository

D7 · Build a custom Nginx image with a Dockerfile (continued)

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations/custom-web`

4 · Optional: publish your custom build to the approved Quay repository

```
export QUAY_CUSTOM_IMAGE="quay.io/${QUAY_NAMESPACE}/${QUAY_REPOSITORY}:custom-v2"
docker tag cf-custom:v2 "$QUAY_CUSTOM_IMAGE"
docker push "$QUAY_CUSTOM_IMAGE"
```

EXPECTED · The approved private repository has the custom-v2 image reference.

PAUSE / SAFETY · Run only after D5 login and validated D6 QUAY_NAMESPACE / QUAY_REPOSITORY values for an existing approved private repository. Write permission and private visibility are not verified by this guide. Skip this block when unavailable; no student sign-up is required.

D8 · Clean up only the resources created in these exercises

TERMINAL: lab host, Ubuntu/WSL Bash **WORKING DIRECTORY:** `$HOME/container-foundations/custom-web`

Before you start: Activities are finished. The instructor confirms which dedicated resources were created and that the data is disposable.

1 · Stop and remove the final web container

```
docker stop cf-custom-web
docker rm cf-custom-web
```

EXPECTED · The final exercise container is removed.

Earlier exercises removed their own instances. Do not remove resources from skipped steps or other learners.

2 · Remove only the dedicated class volume and image tags

DATA DELETION WARNING · inspect your own resources first

Removing `cf-html` deletes D3's persistent page. Confirm the data is disposable and the listed class image tags are yours. Keep the base image for other Workshop tasks. No pruning, forced mass deletion or deletion of the entire class folder.

```
docker volume rm cf-html
docker image rm cf-nginx:archive cf-custom:v1 cf-custom:v2
```

EXPECTED · Only the named exercise volume and selected training image references are removed.

If the optional snapshot exists, remove only `cf-nginx:snapshot` separately under instructor guidance.

OPTIONAL · approved registry-demo participant only

3 · Optional: end the registry session

```
docker logout quay.io
```

EXPECTED · Local Quay authentication is ended.

Logout does not delete remote content; remote cleanup belongs to the repository owner. Source files and the TAR archive also remain in the class folder.

Checkpoint answers

Checkpoint answers and class validation

Optional checkpoint answers

Use these after discussing the three interaction checkpoints. They are not a separate end-of-chapter lab or course quiz.

Q1 · Answer

b, c and e. The question focuses on ordinary application workloads, not privileged host-management tools.

Q2 · Answer

a and c. Runtime CPU constraints belong to container runtime configuration, not the child-image filesystem recipe.

Q3 · Answer

b. A Dockerfile provides a repeatable, reviewable recipe. Dockerfile syntax is not YAML; commit is a snapshot, not the preferred repeatable application-build workflow.

From a single container to AKS—concept only

An image is a deployable application unit. Orchestration coordinates multiple components and provides scheduling, service access, recovery and scaling. The image/container/storage distinctions practiced here carry forward; this guide does not add an AKS implementation lab.

Pre-class check brief · instructor

Status: all 111 shared-manifest command entries passed Bash syntax review. **No Docker runtime rehearsal on the Workshop lab machine has been claimed.** These printed outcomes remain EXPECTED until the instructor validates the environment.

Validate before class	Required scope
Terminal and engine	Approved Ubuntu/WSL Bash, a reachable Linux Docker server, and curl; stop on permission failures rather than changing privileges.
Image and network access	Verify/pre-pull the selected moving alpine tag; confirm classroom Docker Hub access/rate limits and lab-host ports 8080–8083. If public pull is blocked, pause for only an instructor-provided approved registry copy; no personal accounts.
Storage, archive and build	Rehearse cf-html persistence, optional host-path access, TAR save/load, the Dockerfile build, replacement and scoped cleanup on the selected lab configuration.
Optional private registry	Confirm existing approved PRIVATE repository, account/credential method and write authorization before any instructor-only login or push. Otherwise skip those blocks.